

بسمه تعالی

پروژه درس برنامه نویسی پیشرفته

دانشکده فنی انقلاب اسلامی

دانشگاه ملی مهارت

استاد درس: دکتر اردستانی

نام و نام خانوادگی دانشجو: امیر پارسا علیزاده

شماره دانشجویی: 02220040709015

نکته: در تمام این پروژه ها a آخرین رقم غیر صفر شماره دانشجویی شماست.

گزارش تحلیل کد: شبیه‌ساز آسانسور هوشمند

۱. معرفی کلی پروژه

این برنامه یک شبیه‌ساز تعاملی آسانسور در ساختمانی ۵۰ طبقه است که دریافت درخواست‌های مسافران، تصمیم‌گیری برای جهت حرکت (بر پایه‌ی الگوریتم LOOK)، مدیریت سوار و پیاده‌شدن مسافران با توجه به ظرفیت کابین، و محاسبه‌ی زمان انتظار هر مسافر را مدل‌سازی می‌کند.

۲. دسته‌بندی بخش‌های کد

بخش	کلاس/محدوده	نقش
موجودیت پایه‌ی مسافر	Person	نگهداری مبدأ/مقصد و محاسبه‌ی زمان‌های انتظار و سفر
مدیریت صف هر طبقه	Floor	صف‌های جداگانه برای درخواست‌های بالا و پایین
موجودیت فیزیکی آسانسور	Elevator	موقعیت، جهت، وضعیت درب و حرکت
منطق تصمیم‌گیری	Controller	دیسپچینگ، سوار/پیاده‌کردن، آمار انتظار
مدل کلی ساختمان	Building	نگهداری ۵۰ طبقه + آسانسور + ساعت شبیه‌سازی
رابط کاربری و رندر	ElevatorSimulatorGUI	رسم شفت، پنل درخواست، حلقه‌ی اجرا

۳. تحلیل موجودیت‌های پایه

۳.۱. کلاس Person

```
@dataclass
class Person:
    _id_counter = 1
    origin_floor: int
    destination_floor: int
    request_time: float
    id: int = field(init=False)
    ...
    def __post_init__(self):
        self.id = Person._id_counter
        Person._id_counter += 1
```

اینجا از ترکیب @dataclass و __post_init__ استفاده شده است. چون id باید بعد از ساخت سایر فیلدها و بدون دریافت از ورودی مقداردهی شود، field(init=False) آن را از پارامترهای سازنده حذف می‌کند و __post_init__ مقداردهی واقعی را انجام می‌دهد

۳.۲. ویژگی‌های محاسباتی (property)

```
@property
def direction(self) -> Direction:
    return Direction.UP if self.destination_floor > self.origin_floor else Direction.DOWN
```

```
@property
def wait_time(self) -> Optional[float]:
    if self.board_time is None:
        return None
    return self.board_time - self.request_time
```

direction هرگز به صورت مستقیم ذخیره نمی‌شود، بلکه هر بار از روی مبدا و مقصد محاسبه می‌شود؛ این کار از ناهمخوانی داده (Data Inconsistency) جلوگیری می‌کند، چون اگر مقداری به صورت جداگانه ذخیره می‌شد، امکان قدیمی‌شدن یا فراموشی به‌روزرسانی آن وجود داشت. wait_time هم با بازگرداندن Optional[float] به‌صراحت اعلام می‌کند که تا قبل از سوار شدن مسافر، این مقدار قابل محاسبه نیست

۳.۳. کلاس Floor

```
def pop_waiting(self, direction: Direction) -> Optional[Person]:
    queue = self.waiting_up if direction == Direction.UP else self.waiting_down
    return queue.popleft() if queue else None
```

از deque برای صف انتظار استفاده شده تا popleft() با پیچیدگی زمانی $O(1)$ انجام شود. تفاوت کلیدی اینجا وجود دو صف مجزا (بالا/پایین) به جای یک صف واحد است؛ چون در آسانسور، جهت درخواست تعیین‌کننده‌ی این است که مسافر چه زمانی سوار می‌شود

۴. تحلیل کلاس Elevator

۴.۱. الگوریتم LOOK در _decide_direction

```
def _decide_direction(self) -> Direction:
    ups = [f for f in self.target_floors if f > self.current_floor]
    downs = [f for f in self.target_floors if f < self.current_floor]

    if self.direction == Direction.UP and ups:
```

```

    return Direction.UP
if self.direction == Direction.DOWN and downs:
    return Direction.DOWN

if ups and downs:
    return Direction.UP if (min(ups) - self.current_floor) <= (self.current_floor - max(downs)) else
Direction.DOWN
if ups:
    return Direction.UP
if downs:
    return Direction.DOWN
return Direction.IDLE

```

این متد پیاده‌سازی الگوریتم کلاسیک **LOOK** نسخه‌ی بهینه‌شده‌ی SCAN است: دو شرط اول اولویت را به ادامه‌ی جهت فعلی می‌دهند تا از تغییر جهت غیرضروری (Direction Thrashing) جلوگیری شود — حتی اگر یک درخواست در جهت مخالف نزدیک‌تر باشد. فقط زمانی که در جهت فعلی هیچ درخواستی باقی نمانده باشد، به بخش پایین می‌رسد که نزدیک‌ترین درخواست را در هرکدام از دو جهت مقایسه می‌کند. این رفتار دقیقاً همان چیزی است که آسانسورهای واقعی برای کاهش رفت‌وآمد بی‌مورد پیاده‌سازی می‌کنند.

۴.۲. متد — advance

```

def advance(self, dt: float) -> Optional[int]:
    if self.state == ElevatorState.MOVING:
        ...
    if self.state == ElevatorState.DOOR_OPEN:
        ...
    if self.target_floors:
        ...

```

مشابه متد tick در Controller پروژه‌ی چراغ راهنمایی، اینجا هم یک **FSM سه‌حالت** (IDLE, MOVING, DOOR_OPEN) بر پایه‌ی dt پیاده‌سازی شده که هر فراخوانی، وضعیت را بر اساس زمان سپری‌شده به‌روزرسانی می‌کند. نکته‌ی مهم مقدار بازگشتی تابع است Optional[int]: این متد فقط زمانی عددی غیر از None برمی‌گرداند که آسانسور دقیقاً در همین فراخوانی به یک طبقه رسیده باشد. این الگو همان الگوی last_released_direction است، اما این بار به‌صورت مقدار بازگشتی به‌جای متغیر وضعیت است.

۴.۳. حرکت پیوسته با کلمپ کردن به هدف

```

step = self.speed * dt
if self.direction == Direction.UP:
    self.current_floor = min(self.current_floor + step, target)
    reached = self.current_floor >= target
else:
    self.current_floor = max(self.current_floor - step, target)
    reached = self.current_floor <= target

```

current_floor به جای عدد صحیح، یک مقدار اعشاری است که امکان انیمیشن نرم را فراهم می کند. استفاده از min/max برای کلمپ کردن موقعیت به target تضمین می کند که با هر مقدار dt (حتی مقادیر بزرگ ناشی از افت فریم)، آسانسور هرگز از طبقه ی هدف عبور نکند و به جای آن دقیقاً روی آن بنشیند.

۵. تحلیل کلاس Controller

۵.۱. تفکیک مسئولیت بین Elevator و Controller

Elevator هیچ اطلاعی از صف های Floor ندارد و فقط موقعیت، جهت و لیست مسافران داخل کابین را مدیریت می کند. تصمیم گیری درباره ی این که چه کسی سوار/پیاده شود، در Controller._handle_arrival انجام می شود که هم به Elevator و هم به Floorها دسترسی دارد. این جداسازی (Single Responsibility) باعث می شود بتوان منطق دیسپچینگ را بدون تغییر در فیزیک حرکت آسانسور تغییر داد.

۵.۲. مدیریت پیاده شدن و سوار شدن در یک توقف

```
exiting = [p for p in self.elevator.passengers if p.destination_floor == floor]
for person in exiting:
    person.arrival_time = clock
    self.elevator.passengers.remove(person)
    self.completed.append(person)
```

پیاده شدن همیشه قبل از سوار شدن پردازش می شود تا ظرفیت خالی شده در همان توقف برای مسافران جدید در دسترس باشد — رعایت این ترتیب (Order Dependency) از رد شدن غیرضروری مسافران منتظر جلوگیری می کند.

۵.۳. تعیین جهت سرویس هنگام بی کاری

```
serve_direction = self.elevator.direction
if serve_direction == Direction.IDLE:
    if floor_obj.up_call:
        serve_direction = Direction.UP
    elif floor_obj.down_call:
        serve_direction = Direction.DOWN
```

وقتی آسانسور تازه به حالت بی کار رسیده و مستقیماً روی طبقه ای با درخواست ایستاده (نه در حال عبور)، جهتی برای مقایسه با direction مسافر وجود ندارد؛ این بلوک آن حالت مرزی را با انتخاب اولین صف غیرخالی (اولویت با بالا) پوشش می دهد. بدون این شرط، حلقه ی سوار شدن هرگز اجرا نمی شد چون serve_direction برابر IDLE می ماند.

@property

```
def average_wait_time(self) -> Optional[float]:
    waits = [p.wait_time for p in self.completed if p.wait_time is not None]
    return sum(waits) / len(waits) if waits else None
```

فیلتر `if p.wait_time is not None` عملاً همیشه `True` است چون فقط مسافرانی که هم سوار و هم پیاده شده‌اند وارد `self.completed` می‌شوند؛ اما وجود این شرط، تابع را در برابر تغییرات آینده (مثلاً افزودن مسافرانی که به دلایلی دیگر از سیستم خارج می‌شوند) مقاوم نگه می‌دارد و از خطای `TypeError` روی `sum` جلوگیری می‌کند.

۶. تحلیل کلاس Building

`Building` به عنوان لایه‌ی هماهنگ‌کننده‌ی سطح بالا عمل می‌کند: طبقات، آسانسور و کنترلر را می‌سازد و ساعت شبیه‌سازی (`clock`) را در خود نگه می‌دارد، نه در `Controller` این تصمیم منطقی است چون ساعت به کل ساختمان تعلق دارد، نه فقط به منطق دیسپچینگ؛ اگر در آینده چند آسانسور اضافه شود، همه باید از یک ساعت مشترک استفاده کنند.

```
def random_request(self) -> Person:
    origin = random.randint(1, self.num_floors)
    destination = random.randint(1, self.num_floors)
    while destination == origin:
        destination = random.randint(1, self.num_floors)
    return self.request(origin, destination)
```

استفاده از حلقه‌ی `while` برای تضمین متفاوت بودن مبدأ و مقصد، به جای یک شرط ساده، یک الگوی رایج برای نمونه‌گیری مجدد تا برآورده شدن قید (`Rejection Sampling`) است

۷. تحلیل کلاس ElevatorSimulatorGUI

۷.۱. نگاشت مختصات طبقه به پیکسل

```
def _center_y(self, floor_value: float) -> float:
    n = self.building.num_floors
    return self.TOP_MARGIN + (n - floor_value + 0.5) * self.FLOOR_HEIGHT
```

این متد تنها نقطه‌ای در کل برنامه است که تبدیل شماره‌ی طبقه (دامنه‌ی منطقی) به مختصات `y` روی صفحه (دامنه‌ی نمایشی) انجام می‌شود. چون `floor_value` می‌تواند اعشاری باشد (موقعیت لحظه‌ای آسانسور)، همین یک متد هم برای رسم خطوط طبقات (با اعداد صحیح) و هم برای رسم کابین در حال حرکت (با اعداد اعشاری) استفاده می‌شود.

۷.۲. تعامل دوطرفه‌ی Canvas و ورودی‌ها

```
def _on_canvas_click(self, event):
    n = self.building.num_floors
    floor = n - int((event.y - self.TOP_MARGIN) // self.FLOOR_HEIGHT)
    floor = max(1, min(n, floor))
    self.origin_spin.set(floor)
```

این متد دقیقاً معکوس `_center_y` عمل می‌کند: مختصات پیکسلی کلیک کاربر را به شماره‌ی طبقه تبدیل می‌کند. عبارت `max(1, min(n, floor))` مقدار محاسبه‌شده را به بازه‌ی معتبر طبقات کلمپ می‌کند تا کلیک در حاشیه‌ی بالا/پایین Canvas باعث ارسال شماره‌ی طبقه‌ی نامعتبر (کمتر از ۱ یا بیشتر از ۵۰) به Spinbox نشود.

۷.۳. حلقه‌ی اصلی و کنترل سرعت شبیه‌سازی

```
def _loop(self):
    dt = 0.05 * self.speed_multiplier.get()
    messages = self.building.tick(dt)
    ...
    self.root.after(50, self._loop)
```

فاصله‌ی زمانی واقعی بین دو فراخوانی (۵۰ میلی‌ثانیه) ثابت است، اما مقدار `dt` که به شبیه‌سازی داده می‌شود با `speed_multiplier` مقیاس می‌شود. این یعنی تغییر اسلایدر سرعت، فرکانس فراخوانی `after` را تغییر نمی‌دهد (که می‌توانست باعث ناهماهنگی رندر شود)، بلکه فقط میزان «زمان شبیه‌سازی‌شده» در هر فریم را تغییر می‌دهد.

۷.۴. تولید خودکار درخواست به‌عنوان تایمر مستقل

```
if self.auto_spawn.get():
    self.auto_spawn_timer += dt
    if self.auto_spawn_timer >= 4.0:
        self.auto_spawn_timer = 0.0
        self._on_random_request()
```

این بلوک از همان الگوی Accumulator Pattern استفاده می‌کند که در `Elevator.advance` دیده شد: یک شمارنده‌ی مستقل که در هر فریم با `dt` جمع می‌شود و در رسیدن به آستانه بازنشانی می‌گردد.